Table of Contents

Introduction 2

Asynchronous Database Communication 3

Expand the Database with Migrations 6

Lazy Loading 11

Demo: Shop Database, CRUD

Author: Baifan Zhou
Some explanations are created with the help of ChatGPT.

Introduction

This demo covers asynchronous database communication, expanding the database schema using migrations, and implementing lazy loading for efficient data retrieval. Here's an overview of what you will learn:

- **Asynchronous Database Communication**:
  - Transition all database operations in the `ItemController` to asynchronous methods, improving application responsiveness and allowing tasks to run concurrently without blocking the main thread.
  - Update methods to use `async` and `await` keywords, and use asynchronous Entity Framework Core methods such as `ToListAsync()`, `FirstOrDefaultAsync()`, `FindAsync()`, and `SaveChangesAsync()`.
- **Expanding the Database with Migrations**:
  - Shift from using `Database.EnsureCreated()` to Entity Framework Core migrations, which provide a robust way to handle schema changes and database updates over time.
  - Install the `EntityFrameworkCore.Design` package and use the `dotnet ef` command-line tool to create and apply migrations, enabling you to manage schema changes and extend your database with new classes such as `Customer`, `Order`, and `OrderItem`.
- **Lazy Loading**:
  - Implement lazy loading to defer the loading of related entities until they are actually needed, which can improve performance and reduce initial data load.
  - Modify your model classes to include `virtual` navigation properties and configure lazy loading proxies by installing the relevant NuGet package and updating the `ItemDbContext` configuration.

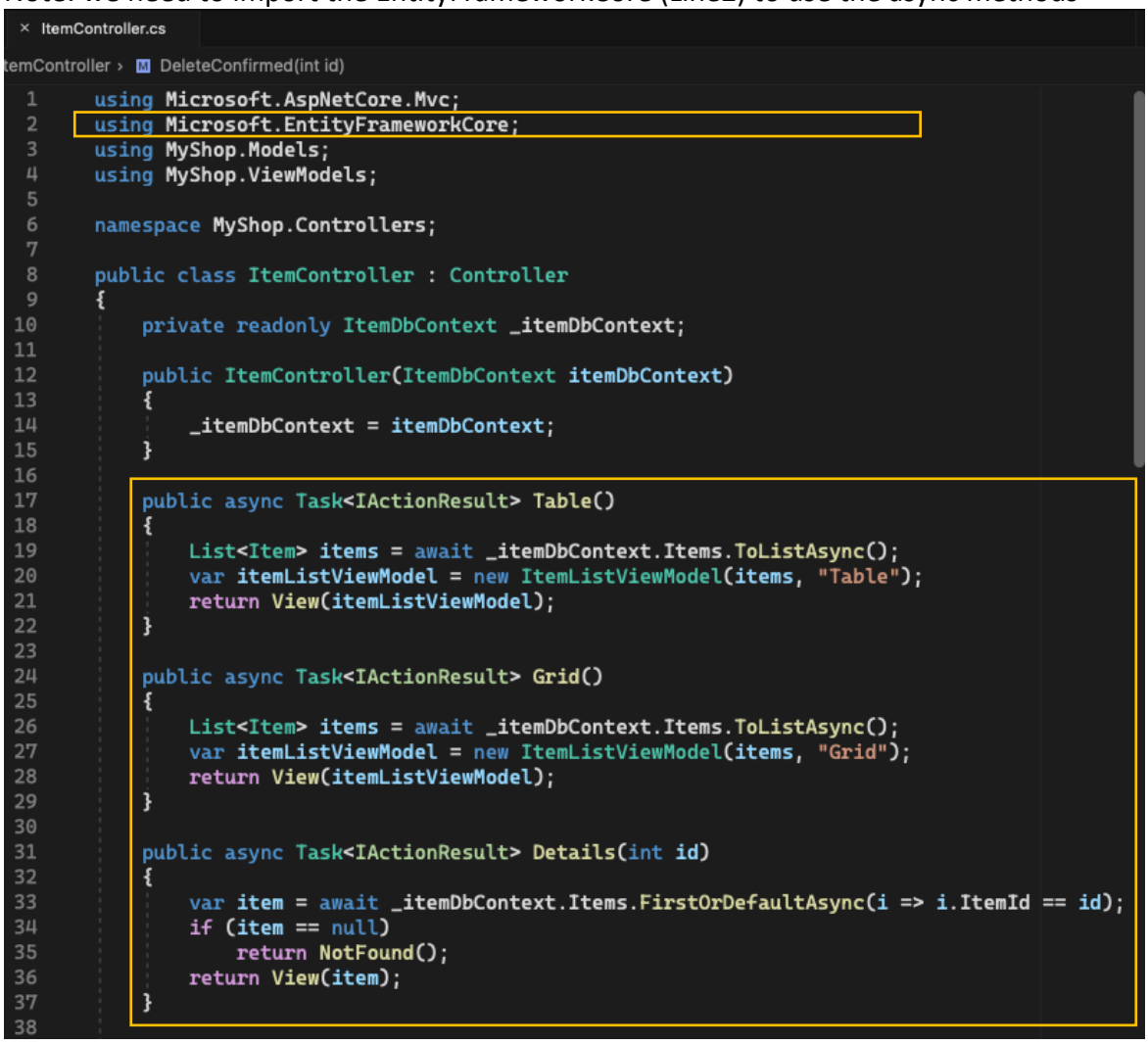
Happy coding!

Asynchronous Database Communication

Async methods allow your application to be more responsive. They allow the application to perform multiple tasks concurrently without blocking the main thread.
In traditional synchronous programming, when a time-consuming task is executed, the entire program execution is halted until that task is completed. During this time, the user interface becomes unresponsive, and the application appears to freeze, leading to a poor user experience.
When an asynchronous task is initiated, the program continues to execute other tasks or respond to user input without waiting for the completion of the time-consuming operation. The asynchronous task runs in the background, and when it is finished, it can notify the application to handle the results.

Now we change our communication with the database to asynchronous. Essentially, we need to change all methods that communicate with the database.

In the Controllers/ItemController.cs, update the methods to be async by adding the async keyword and returning Task<T>:
[HttpGet]: Table(), Grid(), Details(), [HttpGet]Update(), [HttpGet]Delete()
[HtttpPost]: [HtttpPost]Create(), [HtttPost]Update(), [HtttpPost]DeleteConfirmed()
Note: we need to import the EntityFrameworkCore (Line2) to use the async methods



Continue to change the all HttpGet:

```
57      [HttpGet]
58      public async Task<IActionResult> Update(int id)
59      {
60          var item = await _itemDbContext.Items.FindAsync(id);
61          if (item == null)
62          {
63              return NotFound();
64          }
65          return View(item);
66      }
80      [HttpGet]
81      public async Task<IActionResult> Delete(int id)
82      {
83          var item = await _itemDbContext.Items.FindAsync(id);
84          if (item == null)
85          {
86              return NotFound();
87          }
88          return View(item);
89      }
```

Note: The HttpGet version of Create() does not communicate with the database and does not need the async method

```
39      [HttpGet]
40      public IActionResult Create()
41      {
42          return View();
43      }
```

Continue to change the all HttpPost:

```
45      [HttpPost]
46      public async Task<IActionResult> Create(Item item)
47      {
48          if (ModelState.IsValid)
49          {
50              _itemDbContext.Items.Add(item);
51              await _itemDbContext.SaveChangesAsync();
52              return RedirectToAction(nameof(Table));
53          }
54          return View(item);
55      }
68      [HttpPost]
69      public async Task<IActionResult> Update(Item item)
70      {
71          if (ModelState.IsValid)
72          {
73              _itemDbContext.Items.Update(item);
74              await _itemDbContext.SaveChangesAsync();
75              return RedirectToAction(nameof(Table));
76          }
77          return View(item);
78      }
```

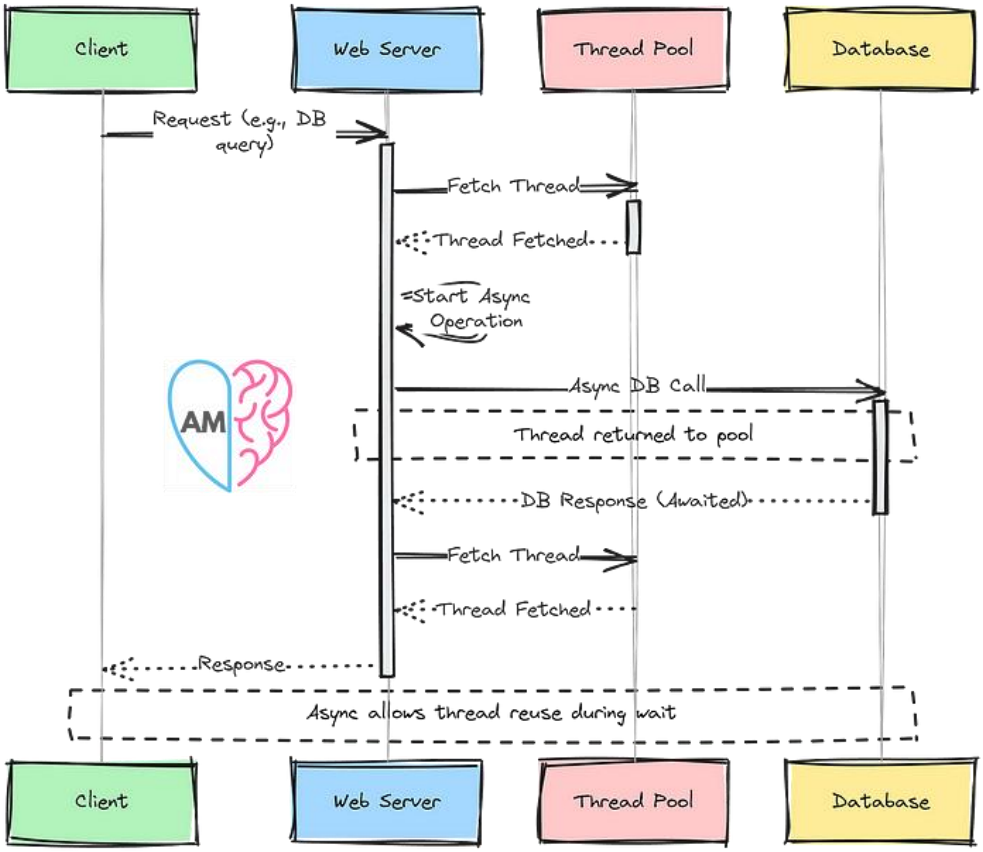
Note:
The Add() method only marks the entity as added in the context and does not immediately persist it to the database. It is therefore not asynchronous. Update() is likewise.
The SaveChangesAsync() method is used to persist all the changes made in the context to the database asynchronously.

```
91      [HttpPost]
92      public async Task<IActionResult> DeleteConfirmed(int id)
93      {
94          var item = await _itemDbContext.Items.FindAsync(id);
95          if (item == null)
96          {
97              return NotFound();
98          }
99          _itemDbContext.Items.Remove(item);
100         await _itemDbContext.SaveChangesAsync();
101         return RedirectToAction(nameof(Table));
102     }
```

All the methods that perform database operations are added an “await” keyword and changed to their async version. These methods include:

- ToListAsync()
- FirstOrDefaultAsync()
- FindAsync()
- SaveChangesAsync()

Read more:
https://medium.com/@vitormoschetta/understanding-synchronous-and-asynchronous-communication-in-software-development-db914a8d7947
https://admirmujkic.medium.com/essential-strategies-and-practices-for-entity-framework-core-595cb7d57782 (Source of the figure below, accessed 30-08-2026)
https://geekbot.com/blog/7-examples-of-asynchronous-communication-at-work-how-to-best-use-them/ (extended reading :)



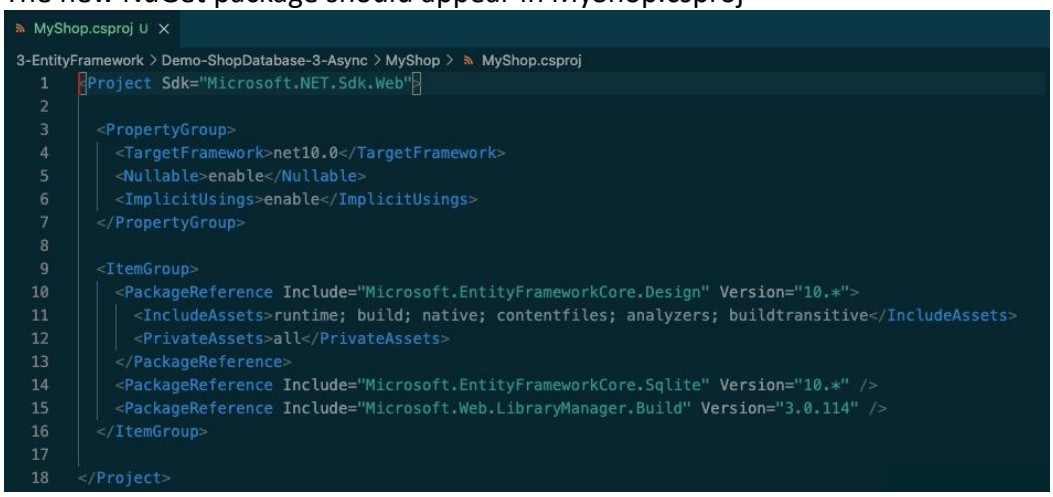
Expand the Database with Migrations

For a shop management system, we need more classes, such as customers, order, etc.
You can image that this update of new classes and properties of classes may also happen in the future.
Previously, we used Database.EnsureCreated() to create a database. It is a simple and convenient way to create a database directly from the DbContext.
It is mainly intended for development and testing scenarios when you want to quickly create a database. It only creates the database schema based on the current model defined in your DbContext. As a result, it doesn't allow for database schema updates or data migrations in future changes to your entities.

Now we want to adopt Migrations, a more robust and scalable way to manage database schema changes over time. Migrations enable you to evolve your database schema and data across multiple application versions.
To do so, we first need to install a NuGet package, EntityFrameworkCore.Design:

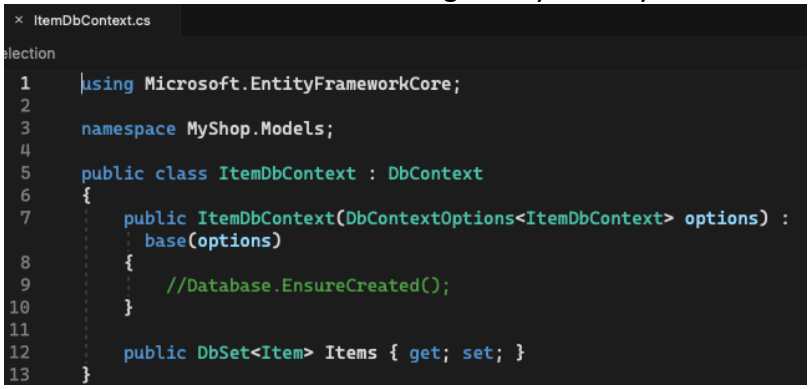
```
dotnet add package Microsoft.EntityFrameworkCore.Design --version="10.*"
```

The new NuGet package should appear in MyShop.csproj



Now **backup** the previous database “ItemDatabase.db” and **DELETE** it from the project.

Comment out the Database.EnsureCreated(); (Line9) in ItemDbContext.cs
This is important! Otherwise, you will get: SQLite Error 1: 'table "Items" already exists'.
This issue typically arises during database migrations or when running code that attempts to create tables without first checking if they already exist.



In the next step, we will create `ItemDatabase.db` using EF Core Migrations.
Open the terminal in your project folder (`MyShop`).

1. Create a tool manifest for your project *(run this once in the project root)*:

```
dotnet new tool-manifest
```

2. Install the `dotnet-ef` tool locally:

```
dotnet tool install dotnet-ef --version "10.*"
```

(Note: Using a local tool avoids modifying system PATH environment variables across Windows, macOS, and Linux.)

3. Verify the EF installation:

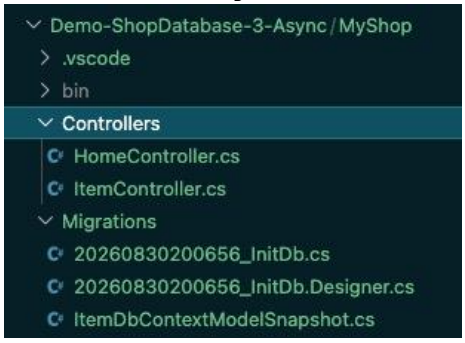
```
dotnet ef -version
```

4. Create the initial migration:

```
dotnet ef migrations add InitDb
```

(Note: If the command fails, rebuild the project first by running `dotnet build`, then try again.)

A folder called `Migrations` should now appear in your project folder:



Then create the ItemDatabase.db with the following terminal command:

```
dotnet ef database update
```

Now the same ItemDatabase.db should appear. Different from the previous version, this one we can expand with more classes.

We now create some classes: Customer, Order, OrderItem, where Customer can have many Orders, one Order consists of multiple OrderItems. One OrderItem specifies which Item is in the Order and its quantity.
Create new classes under Models/ (Pay attention to the non-nullable constraints, string.Empty for strings, and default! For classes.)

Customer.cs

```
election
1   namespace MyShop.Models;
2
3   public class Customer
4   {
5       public int CustomerId { get; set; }
6       public string Name { get; set; } = string.Empty;
7       public string Address { get; set; } = string.Empty;
8       // navigation property
9       public List<Order>? Orders { get; set; }
10  }
```

Order.cs

```
lection
1   namespace MyShop.Models;
2
3   public class Order
4   {
5       public int OrderId { get; set; }
6       public string OrderDate { get; set; } = string.Empty;
7       public int CustomerId { get; set; }
8       // navigation property
9       public Customer Customer { set; get; } = default!;
10      // navigation property
11      public List<OrderItem>? OrderItems { get; set; }
12      public decimal TotalPrice { get; set; }
13  }
```

OrderItem.cs

```
lection
1   namespace MyShop.Models;
2
3   public class OrderItem
4   {
5       public int OrderItemId { get; set; }
6       public int ItemId { get; set; }
7       //navigation property
8       public Item Item { get; set; } = default!;
9       public int Quantity { get; set; }
10      public int OrderId { get; set; }
11      //navigation property
12      public Order Order { get; set; } = default!;
13      public decimal OrderItemPrice { get; set; }
14  }
```

Modify the Item.cs:

Item.cs

```
lection
1   using System.ComponentModel.DataAnnotations;
2
3   namespace MyShop.Models
4   {
5       public class Item
6       {
7           public int ItemId { get; set; }
8           public string Name { get; set; } = string.Empty;
9           public decimal Price { get; set; }
10          public string? Description { get; set; }
11          public string? ImageUrl { get; set; }
12          // navigation property
13          public List<OrderItem>? OrderItems { get; set;}
14      }
15  }
```

Note: List<Order>? Orders is a navigation property in the Customer class. A navigation property in an entity class represents a relationship between entities and allows you to navigate from one entity to related entities.
In this case, the Orders property is a navigation property that represents the relationship between a Customer and their associated Order entities. By having the Orders property in the Customer class, you can easily access all the orders that belong to a specific customer.

Similarly, these navigation properties exist in Order (Customer, OrderItem), OrderItem(Item, Order), Item(OrderItem).

Modify the Models/ItemDbContext.cs to include the newly added classes into the database:

```
ItemDbContext.cs ×
Models > ItemDbContext.cs > ItemDbContext
1   using Microsoft.EntityFrameworkCore;
2
3   namespace MyShop.Models;
4
    7 references
5   public class ItemDbContext : DbContext
6   {
        0 references
7       public ItemDbContext(DbContextOptions<ItemDbContext> options) : base(options)
8       {
9           // Database.EnsureCreated();  // Remove this line if you use migrations
10      }
11
        9 references
12      public DbSet<Item> Items { get; set; }
        0 references
13      public DbSet<Customer> Customers { get; set; }
        0 references
14      public DbSet<Order> Orders { get; set; }
        0 references
15      public DbSet<OrderItem> OrderItems { get; set; }
16  }
```

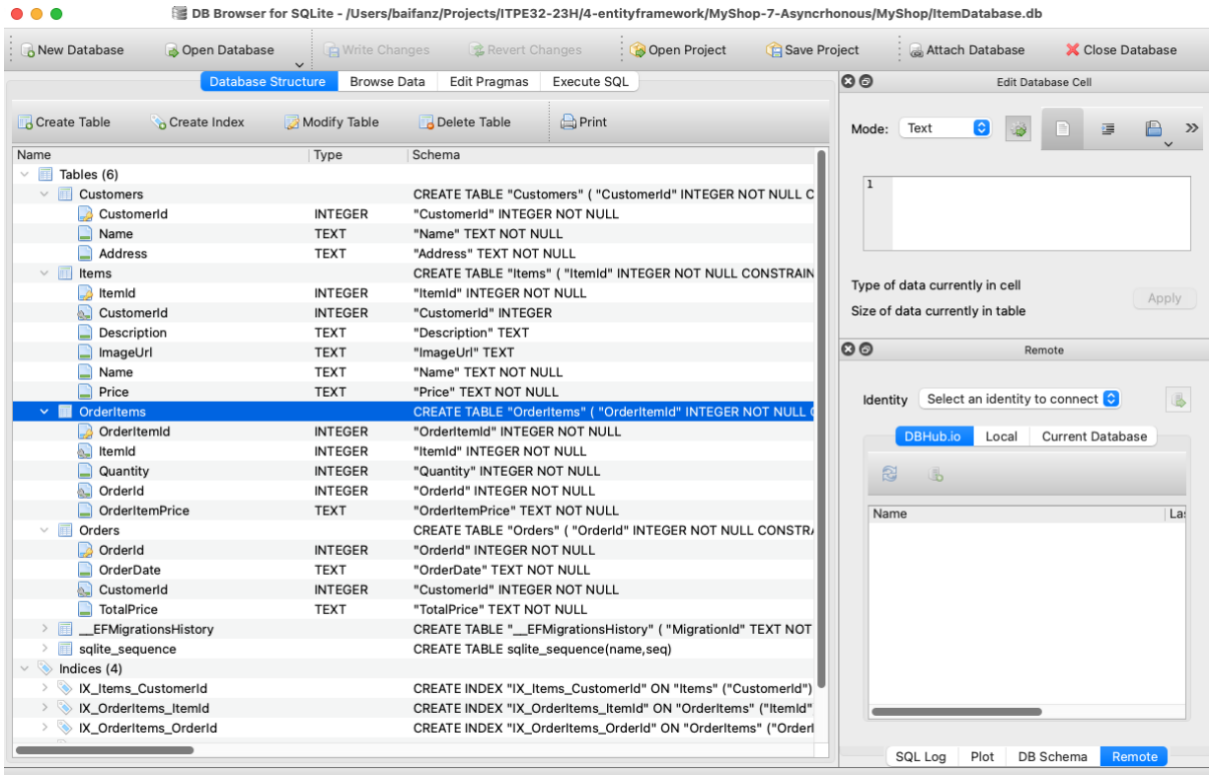
Run the command in Terminal: (rebuild with Command/Control + B in case it fails)

```
dotnet ef migrations add ShopDbExpanded
dotnet ef database update
```

These commands add a new version of Migrations and update the database with the new classes



Open the new ItemDatabase.db with DB Browser for SQLite, you will see the expanded data structure (no data content now because we have not added them)


DB Browser for SQLite - /Users/baifanz/Projects/ITPE32-23H/4-entityframework/MyShop-7-Asyncrhonous/MyShop/ItemDatabase.db
New Database
Open Database
Write Changes
Revert Changes
Open Project
Save Project
Attach Database
Close Database
Database Structure
Browse Data
Edit Pragmas
Execute SQL
Create Table
Create Index
Modify Table
Delete Table
Print
Name
Type
Schema
Tables (6)
Customers
CREATE TABLE "Customers" ("CustomerId" INTEGER NOT NULL C
CustomerId
INTEGER
"CustomerId" INTEGER NOT NULL
Name
TEXT
"Name" TEXT NOT NULL
Address
TEXT
"Address" TEXT NOT NULL
Items
CREATE TABLE "Items" ("ItemId" INTEGER NOT NULL CONSTRAIN
ItemId
INTEGER
"ItemId" INTEGER NOT NULL
CustomerId
INTEGER
"CustomerId" INTEGER
Description
TEXT
"Description" TEXT
ImageUrl
TEXT
"ImageUrl" TEXT
Name
TEXT
"Name" TEXT NOT NULL
Price
TEXT
"Price" TEXT NOT NULL
OrderItems
CREATE TABLE "OrderItems" ("OrderItemId" INTEGER NOT NULL
OrderItemId
INTEGER
"OrderItemId" INTEGER NOT NULL
ItemId
INTEGER
"ItemId" INTEGER NOT NULL
Quantity
INTEGER
"Quantity" INTEGER NOT NULL
OrderId
INTEGER
"OrderId" INTEGER NOT NULL
OrderItemPrice
TEXT
"OrderItemPrice" TEXT NOT NULL
Orders
CREATE TABLE "Orders" ("OrderId" INTEGER NOT NULL CONSTR
OrderId
INTEGER
"OrderId" INTEGER NOT NULL
OrderDate
TEXT
"OrderDate" TEXT NOT NULL
CustomerId
INTEGER
"CustomerId" INTEGER NOT NULL
TotalPrice
TEXT
"TotalPrice" TEXT NOT NULL
__EFMigrationsHistory
CREATE TABLE "__EFMigrationsHistory" ("MigrationId" TEXT NOT
sqlite_sequence
CREATE TABLE sqlite_sequence(name,seq)
Indices (4)
IX_Items_CustomerId
CREATE INDEX "IX_Items_CustomerId" ON "Items" ("CustomerId")
IX_OrderItems_ItemId
CREATE INDEX "IX_OrderItems_ItemId" ON "OrderItems" ("ItemId"
IX_OrderItems_OrderId
CREATE INDEX "IX_OrderItems_OrderId" ON "OrderItems" ("Orderl
Edit Database Cell
Mode:
Text
1
Type of data currently in cell
Size of data currently in table
Apply
Remote
Identity
Select an identity to connect
DBHub.io
Local
Current Database
Name
SQL Log
Plot
DB Schema
Remote

Lazy Loading

In ORM (Object-Relational Mapping) frameworks like Entity Framework Core (EF Core), a technique called lazy loading can defer the loading of related entities until they are accessed. It allows you to load related data from the database on-demand, rather than loading all related data eagerly upfront. This can lead to more efficient data retrieval and improved performance, as only the required data is fetched when needed.

To enable lazy loading, first we add the keyword “virtual” before all the navigation properties: Customer(List<Order>), Item(List<OrderItem>), Order(Customer, List<OrderItem), OrderItem(Item, Order).

Customer.cs

```cs
namespace MyShop.Models;

public class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
    // navigation property
    public virtual List<Order>? Orders { get; set; }
}
```

Item.cs

```cs
using System.ComponentModel.DataAnnotations;

namespace MyShop.Models
{
    public class Item
    {
        public int ItemId { get; set; }
        public string Name { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public string? Description { get; set; }
        public string? ImageUrl { get; set; }
        // navigation property
        public virtual List<OrderItem>? OrderItems { get; set;}
    }
}
```

Order.cs

```cs
namespace MyShop.Models;

public class Order
{
    public int OrderId { get; set; }
    public string OrderDate { get; set; } = string.Empty;
    public int CustomerId { get; set; }
    // navigation property
    public virtual Customer Customer { set; get; } = default!;
    // navigation property
    public virtual List<OrderItem>? OrderItems { get; set; }
    public decimal TotalPrice { get; set; }
}
```

OrderItem.cs

```cs
namespace MyShop.Models;

public class OrderItem
{
    public int OrderItemId { get; set; }
    public int ItemId { get; set; }
    //navigation property
    public virtual Item Item { get; set; } = default!;
    public int Quantity { get; set; }
    public int OrderId { get; set; }
    //navigation property
    public virtual Order Order { get; set; } = default!;
    public decimal OrderItemPrice { get; set; }
}
```

Then we install the NuGet package proxies by running:
`dotnet add package Microsoft.EntityFrameworkCore.Proxies --version="10.*"`
This will appear in MyShop.csproj

MyShop.csproj U

3-EntityFramework > Demo-ShopDatabase-3-Async > MyShop > MyShop.csproj

```
1  <Project Sdk="Microsoft.NET.Sdk.Web">
2
3    <PropertyGroup>
4      <TargetFramework>net10.0</TargetFramework>
5      <Nullable>enable</Nullable>
6      <ImplicitUsings>enable</ImplicitUsings>
7    </PropertyGroup>
8
9    <ItemGroup>
10     <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="10.*">
11       <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</
         IncludeAssets>
12       <PrivateAssets>all</PrivateAssets>
13     </PackageReference>
14     <PackageReference Include="Microsoft.EntityFrameworkCore.Proxies" Version="10.*" />
15     <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.*" />
16     <PackageReference Include="Microsoft.Web.LibraryManager.Build" Version="3.0.114" />
17   </ItemGroup>
18
19 </Project>
```

Now add the Line17 – Line19 to ItemDbContext to enable lazy loading (this requires the new NuGet package “proxies”).

ItemDbContext.cs

lection

```
1   using Microsoft.EntityFrameworkCore;
2
3   namespace MyShop.Models;
4
5   public class ItemDbContext : DbContext
6   {
7       public ItemDbContext(DbContextOptions<ItemDbContext> options) :
          base(options)
8       {
9           Database.EnsureCreated();
10      }
11
12      public DbSet<Item> Items { get; set; }
13      public DbSet<Customer> Customers { get; set; }
14      public DbSet<Order> Orders { get; set; }
15      public DbSet<OrderItem> OrderItems { get; set; }
16
17      protected override void OnConfiguring(DbContextOptionsBuilder
          optionsBuilder)
18      {
19          optionsBuilder.UseLazyLoadingProxies();
20      }
21  }
```

By calling UseLazyLoadingProxies() in the OnConfiguring method, EF Core will generate proxies for your entity classes, allowing lazy loading of related entities when accessed.

With lazy loading enabled, when you access a navigation property like OrderItems of the Order class, EF Core will automatically load the related OrderItem entities from the database on-demand, without explicitly loading the entire tree of dependent objects connected by the navigation properties.

It's important to note that enabling lazy loading comes with certain considerations, such as the potential for increased database queries if navigation properties are accessed in a loop. Be cautious about using lazy loading in performance-critical scenarios and consider using

eager loading (.Include) or explicit loading (DbContext.Entry) when you need to fetch related data in one go.

Read More:
https://learn.microsoft.com/en-us/ef/core/querying/related-data/

Now Run the project. Everything works! ☺ But nothing is to see! No worries, we will verify the proxies and add massive data via database initialisation in the next demo.